COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE 21S2
DATE PERFORMED:	August 07, 2026	**DATE SUBMITTED:**	August 07, 2026
NAME:	Mark Joneil Bautista	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 5 **Queues**

1. PROCEDURE OUTPUT

ILO A:
Code:

```cpp
#include<iostream>
#include<queue>

//prototype declaration
void display(std::queue<char> copyQ);

int main() {

    //create an object:
    std::queue<char> myQ;

    //use the enquque operation
    myQ.push('M');
    myQ.push('A');
    myQ.push('R');
    myQ.push('K');
    std::cout<<"current front is:"<<myQ.front()<<std::endl;
    std::cout<<"current back is:"<<myQ.back()<<std::endl;
    display(myQ);



    //use the dequeue operation

    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);



    return 0;
```

```
35  }
36
37
38  void display(std::queue<char> copyQ){
39      //create a copy of the queue
40      std::queue<char> temp = copyQ;
41      //loop until empty
42      while(!temp.empty()){
43          //display the front
44          std::cout <<" "<< temp.front()<< " ";
45          //dequeue the front
46          temp.pop();
47      }
48      //add a new line
49      std::cout<<std::endl;
50      }
```

Output:

```
current front is:M
current back is:K
 M  A  R  K
 A  R  K
 R  K
 K
```

Code Analysis:
In this program demonstrates the queue operations using std::queue . The push() function enqueues characters ('M', 'A', 'R', 'K') into myQ. The display() function prints the elements without modifying the original queue. Calls to pop() dequeue elements from the front following First In, First Out order.

ILO B.1

Code:

```cpp
#include<iostream>
#include "queueLL.h"

int main() {
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    // Check if empty at the start
    if (isEmpty(front)) {
        std::cout << "Queue is empty at the start.\n";
    }

    enqueue('M', &front, &back);
    std::cout<<"front: "<< front->data<<" " << "back: "<< back->data << "\n";
    enqueue('A', &front, &back);
    std::cout<<"front: "<< front->data<<" " << "back: "<< back->data << "\n";
    enqueue('R', &front, &back);
    std::cout<<"front: "<< front->data<<" " << "back: "<< back->data << "\n";
    enqueue('K', &front, &back);
    std::cout<<"front: "<< front->data<<" " << "back: "<< back->data << "\n";

    // CALL DISPLAY ALL HERE:
    std::cout << "Current Queue: ";
    displayAll(front);

    std::cout<<"testing the dequeue operator: \n";
    dequeue(&front, &back);
    std::cout<<"front: "<< front->data<<" " << "back: "<< back->data << "\n";
    dequeue(&front, &back);
    std::cout<<"front: "<< front->data<<" " << "back: "<< back->data << "\n";
    dequeue(&front, &back);
    std::cout<<"front: "<< front->data<<" " << "back: "<< back->data << "\n";
    dequeue(&front, &back);

    // Check if empty at the end
    if (isEmpty(front)) {
        std::cout << "Queue is empty again.\n";
    }

    dequeue(&front, &back);

    return 0;
}
```

Output:

```
Queue is empty at the start.
front: M back: M
front: M back: A
front: M back: R
front: M back: K
Current Queue: M A R K
testing the dequeue operator:
front: A back: K
front: R back: K
front: K back: K
Queue is empty again.
Queue is empty
```

Code Analysis:
In this code implements a pointer based queue using queueLL.h. The isEmpty() function verifies when the front pointer is nullptr. Calling enqueue() inserts nodes at the back and updates pointers. Calling dequeue() removes nodes from the front until the queue is empty.

ILO B.2

Code:

```cpp
#include <iostream>
#include "queueArr.h"

int main() {
    // creating an object
    QueueArr<int> q(5);

    // enqueue 10, 20, 30, 40, 50:
    q.Enqueue(1);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(2);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(3);
    q.Enqueue(4);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(5);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(6); // Will display "Queue is full"

    std::cout << "\nremoved: " << q.Dequeue() << std::endl;
    std::cout << "removed: " << q.Dequeue() << std::endl;

    q.clear();

    std::cout << "is the queue empty? " << (q.Empty() ? "yes" : "no") << std::endl;

    return 0;
}
```

Output:

```
Front: 1
Back: 1
Size: 1
Front: 1
Back: 2
Size: 2
Front: 1
Back: 4
Size: 4
Front: 1
Back: 5
Size: 5
Queue is full

removed: 1
removed: 2
is the queue empty? yes
```

Code Analysis:
In this code uses an array based queue defined in queueArr.h with a size limit of 5. Adding a 6th element triggers "Queue is full" message. Dequeue() removes the elements from the front of the array. Clear() resets the queue state back to empty.

2. SUPPLEMENTARY OUTPUT

Supplementary
Code:

```cpp
#include <iostream>
#include <string>

class Job {
private:
    int jobID;
    std:: string userName;
    int numberOfPages;

public:
    Job() : jobID(0), userName(""), numberOfPages(0) {}

    Job(int id, std::string name, int pages)
        : jobID(id), userName(name), numberOfPages(pages) {}

    int getJobID() const { return jobID; }
    std::string getUserName() const { return userName; }
    int getNumberOfPages() const { return numberOfPages; }

    void printDetails() const {
        std::cout <<"[Job ID: " << jobID
                  <<" | User: " << userName
                  <<" | Pages: " << numberOfPages << "]";
    }

};

struct Node {
    Job data;
    Node* next;
    Node(Job j) : data(j), next (nullptr) {}
};

class Printer {
```

```
35      private:
36      Node* frontPtr;
37      Node* backPtr;
38      int jobCount;
39
40  public:
41      Printer() : frontPtr(nullptr), backPtr(nullptr), jobCount(0) {}
42
43      ~Printer() {
44          while (!isEmpty()) {
45              processNextJob();
46          }
47      }
48
49      bool isEmpty() const {
50          return frontPtr == nullptr;
51      }
52
53  void addJob(const Job& newJob) {
54      Node* newNode = new Node(newJob);
55      if (isEmpty()) {
56          frontPtr = newNode;
57          backPtr = newNode;
58      } else {
59          backPtr->next = newNode;
60          backPtr = newNode;
61      }
62      jobCount++;
63      std::cout << "-> Added to Spooler: ";
64      newJob.printDetails();
65      std::cout << "\n";
66  }
```

```
67
68  void processNextJob() {
69      if(isEmpty()) {
70          std::cout << "Printer status: No pending jobs to process \n";
71          return;
72      }
73      Node* tempPtr = frontPtr;
74      std::cout << "=> Processing: ";
75      tempPtr->data.printDetails();
76      std::cout << " ... Done!\n";
77
78      frontPtr = frontPtr->next;
79      if (frontPtr == nullptr) {
80          backPtr = nullptr;
81      }
82
83      delete tempPtr;
84      jobCount--;
85  }
86
87  void displayPendingJobs() const {
88      if(isEmpty()) {
89          std::cout << "Printer Queue Status: Empty \n";
90          return;
91      }
92
93      std::cout << "\n--- Current Pending Jobs in Printer Queue (" << jobCount<< ") ---\n"
94      Node* current = frontPtr;
95      int position = 1;
96      while (current != nullptr) {
97          std::cout << position++ << ". ";
```

```
98          current->data.printDetails();
99          std::cout << "\n";
100         current = current->next;
101     }
102     std::cout << "----------------------------------------\n\n";
103 }
104 };
105
106 int main() {
107     Printer officePrinter;
108
109     std::cout << "=== OFFICE SHARED PRINTER ===\n\n";
110     officePrinter.addJob(Job(101, "Alice", 15));
111     officePrinter.addJob(Job(102, "Bob", 3));
112     officePrinter.addJob(Job(103, "Charlie", 42));
113
114     officePrinter.displayPendingJobs();
115
116     std::cout << "--- Printer begins processing ---\n";
117     officePrinter.processNextJob();
118     officePrinter.processNextJob();
119
120
121     officePrinter.processNextJob();
122
123     std::cout << "--- New Job sumbitted while printing ---\n";
124     while (!officePrinter.isEmpty()) {
125         officePrinter.processNextJob();
126     }
127
128     officePrinter.displayPendingJobs();
129
130     return 0;
131
132
133 }
```

Output:

```
=== OFFICE SHARED PRINTER ===

-> Added to Spooler: [Job ID: 101 | User: Alice | Pages: 15]
-> Added to Spooler: [Job ID: 102 | User: Bob | Pages: 3]
-> Added to Spooler: [Job ID: 103 | User: Charlie | Pages: 42]

--- Current Pending Jobs in Printer Queue (3) ---
1. [Job ID: 101 | User: Alice | Pages: 15]
2. [Job ID: 102 | User: Bob | Pages: 3]
3. [Job ID: 103 | User: Charlie | Pages: 42]
----------------------------------------

--- Printer begins processing ---
=> Processing: [Job ID: 101 | User: Alice | Pages: 15] ... Done!
=> Processing: [Job ID: 102 | User: Bob | Pages: 3] ... Done!
=> Processing: [Job ID: 103 | User: Charlie | Pages: 42] ... Done!
--- New Job sumbitted while printing ---
Printer Queue Status: Empty
```

Code Analysis:
This program models a printer spooler using a linked list queue. The Job class stores print job details. The Printer class enqueues jobs using addJob(). Jobs are processed and deleted in FIFO order using processNextJob().

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Problem solving guidance and code verification**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used solely as a learning assistant to explain the mechanics of queues and clarify the activity requirement. I developed, implemented and tested all the C++ code for the array, linked list and STL implementations independently. The AI did not replace my original thinking, code development or analysis.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.